Enhancing Financial Document Retrieval through Domain-Specific Retrieval-Augmented Generation (RAG) Architectures

AI Engineering and Data Science Lab

April 29, 2026

Abstract

Large Language Models (LLMs) suffer from hallucinations and knowledge cutoffs, rendering them unreliable for dynamic, high-stakes tasks such as financial analysis and legal trade document review. Retrieval-Augmented Generation (RAG) bridges this gap by grounding language generation in external, verified knowledge bases. This paper presents a comprehensive pipeline for a domain-specific RAG system, analyzing the mathematical foundations of vector embeddings, optimizing chunking strategies, and evaluating the efficiency of different vector databases in high-dimensional semantic search.

1 Introduction

The explosion of unstructured textual data in economics and finance necessitates automated, highly accurate information retrieval systems. While LLMs excel at reasoning, their parametric memory is static. RAG architecture dynamically retrieves relevant document chunks from a predefined corpus and injects them into the model's context window.

2 System Architecture

2.1 Semantic Embeddings

Textual data must be transformed into continuous vector representations. An embedding function $\mathcal{E}$ maps a text sequence T to a high-dimensional vector $v \in \mathbb{R}^d$.

$$v = \mathcal{E}(T) \tag{1}$$

For domain-specific tasks, models like 'bge-large' or fine-tuned 'sentence-transformers' are utilized. The similarity between a user query vector q and a document chunk vector d is computed using Cosine Similarity:

$$\text{sim}(q, d) = \cos(\theta) = \frac{q \cdot d}{\|q\|\|d\|} = \frac{\sum_{i=1}^{d} q_i d_i}{\sqrt{\sum_{i=1}^{d} q_i^2}\sqrt{\sum_{i=1}^{d} d_i^2}} \tag{2}$$

2.2 Vector Databases and Indexing

To scale the retrieval process across millions of documents, specialized vector databases are employed. Approximate Nearest Neighbor (ANN) search algorithms, specifically Hierarchical Navigable Small World (HNSW) graphs, are implemented to achieve sub-millisecond retrieval latency, balancing recall and query speed.

3 Optimization Strategies

3.1 Advanced Chunking

Naive character-based chunking frequently severs semantic context. We implement a recursive character text splitter with a 15% overlap, optimizing for the preservation of complex financial clauses and mathematical formulas.

3.2 Re-ranking Mechanisms

Initial retrieval via cosine similarity is fast but often lacks deep contextual understanding. We introduce a secondary cross-encoder re-ranking step. Given a query q and a retrieved document d, the cross-encoder computes a relevance score $S_{rel}(q, d)$ to re-order the top-K candidates before context injection.

4 Conclusion

Building a robust RAG pipeline requires meticulous tuning of embedding models, vector indexing, and retrieval strategies. By optimizing these components, the reliability of AI-driven financial document analysis is drastically improved.